

Python 面向对象编程



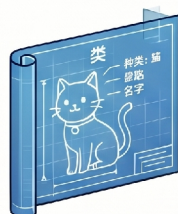
学习目标：

1. 理解面向对象编程（OOP）的基本思想，区分面向对象与面向过程。
2. 掌握类和对象的概念，能够定义类和创建对象。
3. 理解封装、继承、多态三大特征，并能编写示例。
4. 区分实例属性/实例方法、类属性/类方法、静态方法的使用场景。
5. 掌握常见的魔法方法（`__init__`、`__str__`、`__repr__`、`__len__`、`__add__` 等）。
6. 理解 Python 中的鸭子类型（Duck Typing）及其灵活性。
7. 能够综合运用面向对象知识解决实际问题。



Python 面向对象编程 (OOP) 核心指南

基础概念：从“图纸”到“实例”



类 (Class)
[图纸]



构造方法 `__init__`

对象创建时自动调用的“装修”环节，用于初始化对象的属性，且不能有返回值。



对象 (Object)
[实例]



核心支柱：三大特征与 Python 灵活性



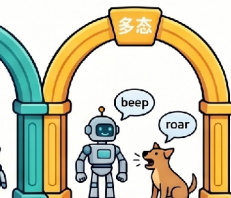
封装

隐藏细节（私有属性 `__`）



继承

实现代码复用



多态

同一方法名在不同对象中产生不同行为

三类方法的应用场景

实例方法

第一个参数: `self`
访问权限: 实例属性+类属性
调用方式: 对象.方法()



类方法

第一个参数: `cls`
访问权限: 仅限类属性
调用方式: 类名.方法()



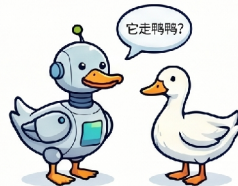
静态方法

第一个参数: 无
访问权限: 均不可访问
调用方式: 类名.方法()



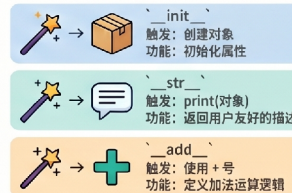
鸭子类型

Python 的核心风格：关注“它走起来像不像鸭子”，而不关心它具体的类型。



魔法方法

以双下划线开头结尾，让自定义对象能像内置数据类型一样进行操作。



第十三章：面向对象基础

1. 为什么需要面向对象？

定义：面向对象编程（Object-Oriented Programming, OOP）是一种编程范式，它把现实世界中的事物抽象为“对象”，通过对象之间的交互来设计程序。

生活类比：面向过程编程像是做菜步骤清单（切菜 → 炒菜 → 装盘），适合简单的任务。面向对象编程像是把厨房里的东西都看作对象（锅、灶台、菜刀、食材），每个对象有自己的属性和行为，你指挥它们协同工作。对于复杂的系统（如游戏、电商平台），面向对象让代码更清晰、更易扩展。

好处：

- 更贴近人类认识世界的方式（把数据和行为绑定在一起）。
- 代码重用性高（继承、组合）。
- 维护方便，修改一个类不影响其他部分。
- 适合大型项目团队协作。

2. 类与对象

2.1 什么是类？什么是对象？

定义：

- **类 (Class)：**是对一类具有相同属性和行为的事物的抽象描述。可以理解为“图纸”或“模板”。
- **对象 (Object)：**是根据类创建出来的具体实例。可以理解为“根据图纸造出来的真实房子”。

大白话：

- 类就是“猫”这个类别，它有属性（毛色、年龄）和行为（叫、抓老鼠）。
- 对象就是具体的某只猫，比如“我家的大橘”。

示例：定义类并创建对象。

```
1 # 定义一个类（图纸）
2 class Cat:
3     # 属性（特征）和 方法（行为）稍后添加
4     pass
5
6 # 根据类创建对象
7 cat1 = Cat() # cat1 是一个对象（实例）
8 cat2 = Cat()
9 print(type(cat1)) # <class '__main__.Cat'>
```

2.2 实例属性与实例方法

● 实例属性

定义：属于具体对象的属性，不同对象的实例属性可以不同。通常在 `__init__` 方法中通过 `self.属性名` 定义。

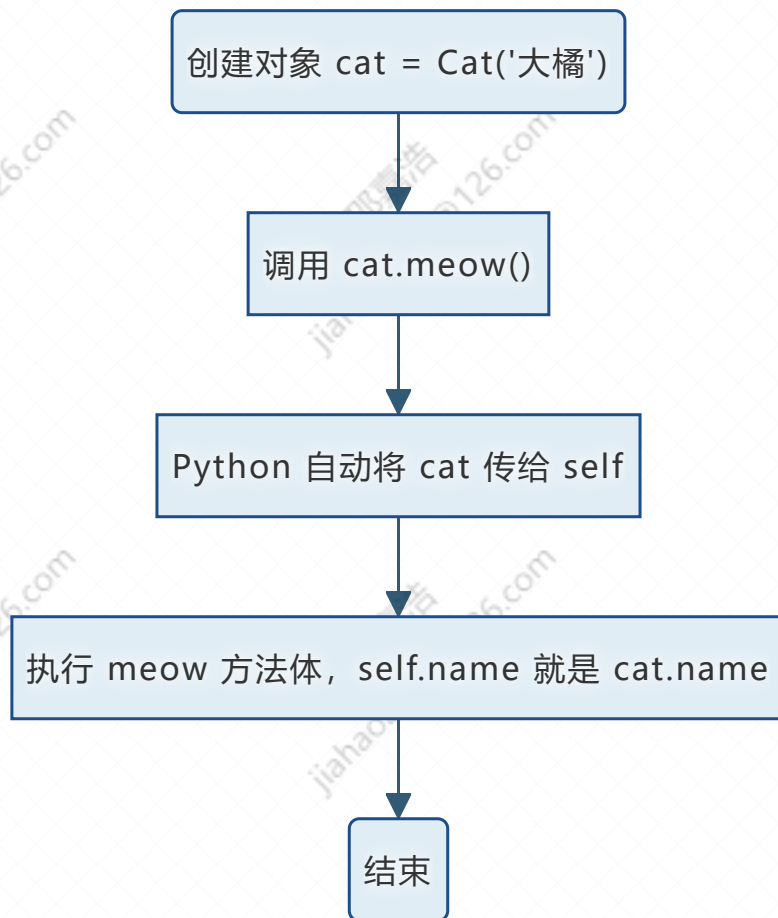
```
1 class Cat:
2     def __init__(self, name, color):
3         # 实例属性
4         self.name = name # 每只猫有自己的名字
5         self.color = color # 每只猫有自己的颜色
6
7 # 创建对象时传入初始值
8 cat1 = Cat("大橘", "橙色")
9 cat2 = Cat("小白", "白色")
10 print(cat1.name) # 大橘
11 print(cat2.name) # 小白
```

● 实例方法

定义：定义在类中的函数，第一个参数必须是 `self`，表示调用该方法的对象本身。实例方法可以访问和修改实例属性。

```
python
1  class Cat:
2      def __init__(self, name):
3          self.name = name
4
5      def meow(self):
6          print(f"{self.name} 喵喵叫！")
7
8      def sleep(self, hours):
9          print(f"{self.name} 睡了 {hours} 小时")
10
11 cat = Cat("大橘")
12 cat.meow()          # 大橘 喵喵叫！
13 cat.sleep(3)        # 大橘 睡了 3 小时
```

流程图：对象调用实例方法



📌 Important

实例方法的第一个参数必须写 `self`，但调用时不需要手动传递。

2.3 构造方法 `__init__`

定义： `__init__` 是一个特殊的魔法方法，在创建对象时自动调用，用于初始化对象的属性。

大白话：就像买房交房时的“装修”环节，房子（对象）造好后，给它刷上颜色、放好家具（设置初始属性）。

python

```
1 class Student:
2     def __init__(self, name, age):
3         print(f"正在初始化 {name}")
4         self.name = name
5         self.age = age
6
```

```
7 stu = Student("小明", 18) # 自动调用 __init__
8 print(stu.name) # 小明
```

① Note

`__init__` 不能有返回值（返回 `None`）。

2.4 类属性与类方法

● 类属性

定义：属于类本身的属性，所有实例共享同一个类属性。可以直接通过 `类名.属性名` 访问，也可以通过实例访问（但不推荐修改）。

```
python
1 class Cat:
2     # 类属性（所有猫共同的属性）
3     species = "猫科动物"
4
5     def __init__(self, name):
6         self.name = name
7
8     print(Cat.species) # 猫科动物
9     cat1 = Cat("大橘")
10    cat2 = Cat("小白")
11    print(cat1.species) # 猫科动物（实例也可以读取类属性）
12    print(cat2.species) # 猫科动物
```

● 类方法

定义：使用 `@classmethod` 装饰器定义，第一个参数是 `cls`（表示类本身）。类方法可以访问和修改类属性。

```
python
1 class Cat:
```

```

2     species = "猫科动物"
3     total_cats = 0    # 记录创建了多少只猫
4
5     def __init__(self, name):
6         self.name = name
7         Cat.total_cats += 1
8
9     @classmethod
10    def get_total(cls):
11        return cls.total_cats
12
13    @classmethod
14    def change_species(cls, new_species):
15        cls.species = new_species
16
17    print(Cat.get_total())    # 0
18    cat1 = Cat("大橘")
19    cat2 = Cat("小白")
20    print(Cat.get_total())    # 2
21
22    Cat.change_species("Felidae")
23    print(Cat.species)        # Felidae

```

使用场景：工厂方法、统计实例数量、修改类级别的配置。

2.5 静态方法

定义：使用 `@staticmethod` 装饰器定义，不需要 `self` 或 `cls` 参数。它只是恰好定义在类内部的普通函数，与类和实例没有强制绑定。

大白话：静态方法就像一个放在类文件夹里的工具箱，不需要访问属性，只完成独立的功能。



python

```

1     class MathUtils:
2         @staticmethod

```

```

3         def add(a, b):
4             return a + b
5
6         @staticmethod
7         def is_even(n):
8             return n % 2 == 0
9
10    print(MathUtils.add(3, 5))    # 8
11    print(MathUtils.is_even(4))  # True

```

对比总结：

类型	定义方式	第一个参数	访问类属性	访问实例属性	调用方式
实例方法	普通方法	self	可以	可以	对象.方法() 或 类.方法(实例)
类方法	@classmethod 装饰	cls	可以	不可以	类.方法() 或 对象.方法()
静态方法	@staticmethod 装饰	无	不可以	不可以	类.方法() 或 对象.方法()

3. 课堂练习（基础）

1. 定义一个 `Dog` 类，具有实例属性 `name` 和 `age`，实例方法 `bark()` 打印“汪汪”。创建两只狗对象，分别调用 `bark()`。
2. 添加类属性 `species = "犬科"`，类方法 `get_species()` 返回该属性，静态方法 `is_adult(age)` 返回年龄是否大于等于 2。
3. 使用 `__init__` 记录每只狗的创建，并用类属性统计总共创建了多少只狗。

第十四章：面向对象的三大特征

1. 封装

定义：封装（Encapsulation）将数据（属性）和操作数据的方法（行为）捆绑在一起，并隐藏内部实现细节，只对外暴露必要的接口。

大白话：就像手机，你只需要按按钮（接口）就能打电话，不需要知道内部电路如何工作。

Python 中的封装：

- 公有属性/方法：直接命名（`self.name`），外部可访问。
- 约定私有属性/方法：以单下划线开头（`_age`），表示“受保护的”，外部不应直接访问（约定而成）。
- 名称修饰私有成员：以双下划线开头（`__password`），Python 会自动改名，防止子类意外覆盖。



python

```
1 class BankAccount:
2     def __init__(self, owner, balance):
3         self.owner = owner          # 公有属性
4         self._bank = "ICBC"        # 约定私有（受保护）
5         self.__balance = balance   # 名称修饰私有（真实名字为
    _BankAccount__balance）
6
7     def deposit(self, amount):
8         if amount > 0:
9             self.__balance += amount
10
11     def get_balance(self):
12         return self.__balance
13
14 acc = BankAccount("小明", 1000)
15 print(acc.owner)          # 小明
16 print(acc._bank)          # ICBC（虽然能访问，但不建议）
```

```
17 # print(acc.__balance)      # 报错 AttributeError
18 print(acc.get_balance())    # 1000
```

💡 Tip

Python 的“私有”是约定俗成的，不是强制不可访问。真正隐藏细节靠程序员自觉。

2. 继承

定义：继承（Inheritance）允许一个类（子类）继承另一个类（父类）的属性和方法，实现代码复用。

生活类比：孩子继承父母的基因，但也可以有自己的新特征。

基本语法：

```
1 class Animal:
2     def __init__(self, name):
3         self.name = name
4
5     def eat(self):
6         print(f"{self.name} 正在吃东西")
7
8 class Dog(Animal):      # Dog 继承 Animal
9     def bark(self):
10        print(f"{self.name} 汪汪叫")
11
12 dog = Dog("旺财")
13 dog.eat()    # 继承来的方法
14 dog.bark()   # 自己的方法
```

2.1 方法重写（Override）

子类可以重新定义父类的方法。

```

1 class Cat(Animal):
2     def eat(self):          # 重写父类方法
3         print(f"{self.name} 优雅地吃鱼")
4
5 cat = Cat("咪咪")
6 cat.eat()      # 咪咪 优雅地吃鱼

```

2.2 super() 调用父类方法

子类可以在重写的方法中调用父类的版本。

```

1 class Dog(Animal):
2     def __init__(self, name, breed):
3         super().__init__(name)    # 调用父类的 __init__
4         self.breed = breed
5
6     def eat(self):
7         super().eat()              # 先执行父类的 eat
8         print(f"{self.name} 吃完后还舔了舔嘴巴")
9
10 dog = Dog("旺财", "金毛")
11 dog.eat()

```

2.3 多重继承

Python 支持一个子类继承多个父类。

```

1 class Flyable:
2     def fly(self):
3         print("飞起来了")
4
5 class Swimmable:
6     def swim(self):

```

```

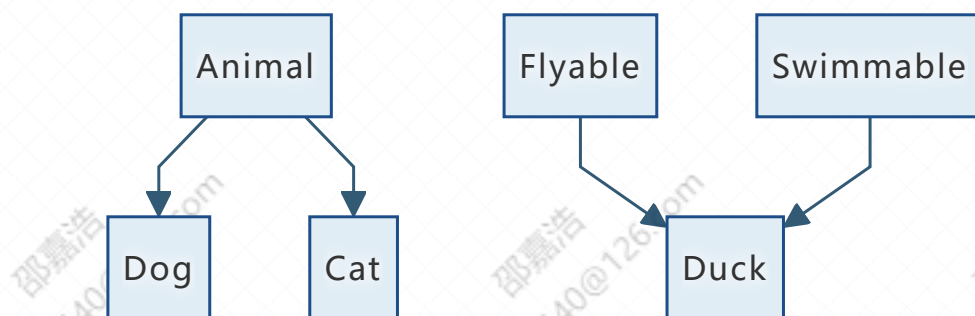
7         print("游起来了")
8
9     class Duck(Flyable, Swimmable):
10         pass
11
12     duck = Duck()
13     duck.fly()    # 飞起来了
14     duck.swim()  # 游起来了

```

① Note

多重继承可能带来“菱形继承”问题，Python 使用 **C3 线性化算法** 解决，初学者应先避免复杂多重继承。

流程图：继承关系



3. 多态

定义：多态（Polymorphism）指同一个方法在不同类中可以有不同实现。调用者不需要关心对象的具体类型，只需要知道它有这个方法。

大白话：你让宠物“叫”，猫会“喵喵”，狗会“汪汪”。你只需要调用 `make_sound()`，具体实现由具体对象决定。

多态与继承的关系：通常多态基于继承，子类重写父类方法，然后使用父类类型的变量指向子类对象。

```

python
1     # 父类
2     class Animal:
3         def speak(self):

```

```

4         pass
5
6     # 子类1
7     class Dog(Animal):
8         def speak(self):
9             print("汪汪汪")
10
11    # 子类2
12    class Cat(Animal):
13        def speak(self):
14            print("喵喵喵")
15
16    # 子类3
17    class Duck(Animal):
18        def speak(self):
19            print("嘎嘎嘎")
20
21    # 统一调用接口（多态核心）
22    def make_sound(animal: Animal):
23        animal.speak()
24
25    # 多态表现：同一个函数，不同对象，不同行为
26    make_sound(Dog())    # 汪汪汪
27    make_sound(Cat())    # 喵喵喵
28    make_sound(Duck())   # 嘎嘎嘎

```

一句话记住多态：同一个方法名，不同对象做不同事，调用方式完全一样。



方法重写是实现多态的手段

多态是方法重写之后产生的效果

4. 课堂练习（三大特征）

1. 设计一个 `Vehicle` 类，有 `start()` 方法（打印“车辆启动”）。子类 `Car` 和 `Bike` 重写 `start()`，分别打印“汽车发动”和“自行车踩踏”。
2. 定义一个 `Person` 类，有私有属性 `__age`，提供 `set_age(age)` 方法（判断 `age` 在 0-150 之间）和 `get_age()` 方法。实现封装。
3. 定义 `Bird` 和 `Fish` 类，都有 `move()` 方法。编写函数 `perform_move(animal)`，传入任意有 `move()` 方法的对象，调用它。演示多态。

第十五章：魔法方法

魔法方法（Magic Method）也叫 双下方法 / 特殊方法：

- 以 双下划线 `__` 开头和结尾 的方法
- 不需要手动调用，Python 在特定时机自动触发
- 作用：自定义类的行为，让自定义对象能像内置对象（`str`、`list`、`dict`）一样使用

1. 常见的魔法方法

分类	魔法方法	触发时机	作用
基础	<code>__init__</code>	创建对象时	初始化属性
	<code>__str__</code>	<code>print(obj)</code>	返回用户友好字符串
	<code>__repr__</code>	交互环境控制台、容器、 <code>repr()</code> 没有定义 <code>__str__</code> 时, <code>print(obj)</code> 会降级用 <code>__repr__</code>	开发调试字符串
	<code>__len__</code>	<code>len(obj)</code>	获取对象长度
运算符	<code>__del__</code>	对象销毁	析构清理
	<code>__add__</code>	<code>+</code>	加法
	<code>__sub__</code>	<code>-</code>	减法
	<code>__mul__</code>	<code>*</code>	乘法
	<code>__truediv__</code>	<code>/</code>	除法
比较	<code>__eq__</code>	<code>==</code>	判断相等
	<code>__gt__</code>	<code>></code>	判断大于
	<code>__getitem__</code>	<code>obj[key]</code>	索引取值
容器	<code>__setitem__</code>	<code>obj[key]=val</code>	索引赋值
	<code>__contains__</code>	<code>item in obj</code>	判断包含
	<code>__iter__</code>	<code>for...in</code>	可迭代
	<code>__call__</code>	<code>obj()</code>	对象当函数用
调用	<code>__bool__</code>	<code>bool(obj)</code>	布尔判断
	<code>__enter__</code>	<code>with</code> 开始	进入上下文

分类	魔法方法	触发时机	作用
	<code>__exit__</code>	<code>with</code> 结束	退出上下文

2. 示例代码

python

```
1  class Student:
2      # 1. 初始化
3      def __init__(self, name, score):
4          self.name = name
5          self.score = score
6
7      # 2. 打印对象
8      def __str__(self):
9          return f"学生: {self.name}, 分数: {self.score}"
10
11     # 3. 调试表示
12     def __repr__(self):
13         return f"Student('{self.name}', {self.score})"
14
15     # 4. 长度
16     def __len__(self):
17         return len(self.name)
18
19     # 5. 加法
20     def __add__(self, other):
21         return self.score + other.score
22
23     # 6. 减法
24     def __sub__(self, other):
25         return self.score - other.score
26
27     # 7. 乘法
28     def __mul__(self, other):
29         return self.score * other.score
```

```
30
31     # 8. 除法
32     def __truediv__(self, other):
33         return self.score / other.score
34
35     # 9. 相等判断
36     def __eq__(self, other):
37         return self.score == other.score
38
39     # 10. 大于判断
40     def __gt__(self, other):
41         return self.score > other.score
42
43     # 11. 索引取值
44     def __getitem__(self, key):
45         return self.__dict__[key]
46
47     # 12. 索引赋值
48     def __setitem__(self, key, value):
49         self.__dict__[key] = value
50
51     # 13. 判断包含
52     def __contains__(self, item):
53         return item in self.__dict__.values()
54
55     # 14. 可迭代
56     def __iter__(self):
57         yield from [self.name, self.score]
58
59     # 15. 对象调用
60     def __call__(self):
61         return f"{self.name}正在被调用"
62
63     # 16. 布尔判断
64     def __bool__(self):
65         return self.score >= 60
```

```

66
67     # 17. 上下文进入
68     def __enter__(self):
69         print("=== 进入 with 语句 ===")
70         return self
71
72     # 18. 上下文退出
73     def __exit__(self, exc_type, exc_val, exc_tb):
74         print("=== 退出 with 语句 ===")
75
76     # 19. 析构方法
77     def __del__(self):
78         pass
79
80
81     # ===== 测试 =====
82     if __name__ == "__main__":
83         s1 = Student("小明", 90)
84         s2 = Student("小红", 80)
85
86         print(s1)                # __str__
87         print(len(s1))           # __len__
88         print(s1 + s2)           # __add__
89         print(s1 > s2)           # __gt__
90         print(s1["name"])        # __getitem__
91         s1["age"] = 20           # __setitem__
92         print("小明" in s1)      # __contains__
93
94         for item in s1:          # __iter__
95             print(item)
96
97         print(s1())              # __call__
98         print(bool(s1))          # __bool__
99
100        with Student("小刚", 85) as s: # __enter__ / __exit__
101            print(s)

```




Tip

`__str__` 和 `__repr__` 通常一起定义。如果只定义 `__str__`，那么 `repr` 会退化为默认的类型+地址。

3. 课堂练习（魔法方法）

1. 定义一个 `Book` 类，属性有 `title`，`author`，`pages`。实现 `__str__` 返回 "`《书名》 by 作者`"，实现 `__len__` 返回页数，实现 `__eq__` 判断两本书是否作者相同。
2. 定义一个 `Vector` 类，表示二维向量 (`x`, `y`)。实现 `__add__` 向量加法，`__sub__` 减法，`__mul__` 标量乘法（如 `v * 3`）。

第十六章：鸭子类型（Duck Typing）

定义：鸭子类型是 Python 等动态语言中的一种编程风格。其核心思想是：“如果它看起来像鸭子，叫起来像鸭子，那么它就是鸭子。”不关心对象的类型，只关心对象是否具有所需的方法或属性。

大白话：你不需要告诉 Python 参数必须是某个类的实例，只要它有你要调用的方法，就可以使用。

示例：

```
1 class Duck:
2     def quack(self):
3         print("嘎嘎")
4
5 class Person:
6     def quack(self):
7         print("我模仿鸭子嘎嘎叫")
8
9 def make_quack(obj):
10     obj.quack()
```

python

```
11
12     duck = Duck()
13     person = Person()
14     make_quack(duck)      # 嘎嘎
15     make_quack(person)   # 我模仿鸭子嘎嘎叫
```

与多态的区别：

- 多态通常基于继承：子类重写父类方法，通过父类引用调用。
- 鸭子类型不要求继承关系，只要对象有同名方法即可。

优点：代码更灵活，减少硬编码的类型检查。

缺点：如果对象没有所需方法，会在运行时抛出 `AttributeError`，调试难度稍高。

第十七章：综合项目 —— 学校人员管理系统（学生 + 老师）

需求：

1. 父类：Person（人）
2. 子类：Student（学生）、Teacher（老师）
3. 封装：私有属性 + 公共方法
4. 继承：子类复用父类
5. 多态：同名方法不同表现
6. 魔法方法： `__init__`、`__str__`、`__repr__`
7. 方法重写

```
1  # 父类：人
2  class Person:
3      def __init__(self, name, age):
4          # 封装：私有属性
5          self.__name = name
6          self.__age = age
```

python

```
7
8     # 公共方法：获取姓名
9     def get_name(self):
10         return self.__name
11
12     # 公共方法：获取年龄
13     def get_age(self):
14         return self.__age
15
16     # 自我介绍（多态基础）
17     def introduce(self):
18         return f"我是{self.__name}, 今年{self.__age}岁"
19
20     # 魔法方法：打印显示
21     def __str__(self):
22         return self.introduce()
23
24     # 魔法方法：调试显示
25     def __repr__(self):
26         return f"Person('{self.__name}', {self.__age})"
27
28
29 # 子类：学生（继承 Person）
30 class Student(Person):
31     def __init__(self, name, age, stu_id, score):
32         # 调用父类构造
33         super().__init__(name, age)
34         self.stu_id = stu_id
35         self.score = score
36
37     # 方法重写（多态）
38     def introduce(self):
39         return f"【学生】我是{self.get_name()}, 学号:
40 {self.stu_id}, 成绩: {self.score}"
41
42     # 学生独有方法
```

```
42     def study(self):
43         return f"{self.get_name()}正在学习..."
44
45
46 # 子类：老师（继承 Person）
47 class Teacher(Person):
48     def __init__(self, name, age, subject, salary):
49         super().__init__(name, age)
50         self.subject = subject
51         self.salary = salary
52
53     # 方法重写（多态）
54     def introduce(self):
55         return f"【老师】我是{self.get_name()}, 教
56 {self.subject}, 月薪: {self.salary}"
57
58     # 老师独有方法
59     def teach(self):
60         return f"{self.get_name()}正在讲课..."
61
62 # ===== 测试 =====
63 if __name__ == '__main__':
64
65     # 创建学生
66     s = Student("小明", 18, "2026001", 95)
67     # 创建老师
68     t = Teacher("张老师", 35, "Python", 15000)
69
70     # 打印（自动触发 __str__）
71     print(s)
72     print(t)
73
74     # 多态：同一个方法，不同表现
75     print("\n=== 多态演示 ===")
76     def show_info(person):
```

```
77         print(person.introduce())
78
79     show_info(s)
80     show_info(t)
81
82     # 调用子类独有方法
83     print("\n=== 独有方法 ===")
84     print(s.study())
85     print(t.teach())
86
87     # 获取私有属性（通过公共方法）
88     print("\n=== 封装演示 ===")
89     print("学生姓名: ", s.get_name())
90     print("老师年龄: ", t.get_age())
```

总结与复习

概念	定义	要点
类	对象的蓝图或模板。	用 <code>class</code> 定义。
对象	根据类创建的具体实例。	<code>对象 = 类名()</code> 。
实例属性	属于具体对象的属性，在 <code>__init__</code> 中用 <code>self</code> 定义。	每个对象独立。
实例方法	定义在类中，第一个参数为 <code>self</code> 的函数。	可以访问实例属性和类属性。
类属性	属于类本身的属性，所有实例共享。	通过 <code>类名.属性</code> 访问或修改。
类方法	<code>@classmethod</code> ，第一个参数 <code>cls</code> 。	可以访问类属性，不能访问实例属性。
静态方法	<code>@staticmethod</code> ，无特殊参数。	独立功能，不依赖类和实例。
封装	隐藏内部实现，对外提供接口。	私有属性用双下划线 <code>__</code> 进行名称修饰。
继承	子类复用父类的属性和方法。	用 <code>class 子类(父类):</code> ，支持多重继承。
多态	同一方法在不同类中有不同实现。	依赖继承和方法重写。
鸭子类型	不检查类型，只检查是否有需要的方法。	提高灵活性，但可能运行时出错。
魔法方法	双下划线开头结尾的方法，自动触发。	<code>__init__</code> ， <code>__str__</code> ， <code>__add__</code> 等。

课后作业

1. 汽车租赁系统

定义 `Car` 类，属性：车牌号、品牌、日租金、是否租出。方法：`rent()` 租车、`return_car()` 还车。定义 `Customer` 类，属性：姓名、驾照号、租车历史列表。实现简单的租车交互。

2. 图形面积计算

定义抽象类 `Shape`（可以使用 `abc` 模块，或约定子类必须实现 `area()` 方法）。子类 `Circle`，`Rectangle`，`Triangle` 分别实现 `area()`。多态地计算一组图形的总面积。

3. 魔法方法练习

实现一个 `Fraction` 分数类，属性 `numerator`（分子）和 `denominator`（分母）。实现 `__add__` 分数加法、`__eq__` 判断相等、`__str__` 输出如 `"2/3"`。